

**LAPORAN PROJECT BASED LEARNING
ANALISIS LOG SERVER/WEB TRAFFIC**

Disusun Untuk Memenuhi Evaluasi Akhir Semester
Mata Kuliah Big Data



DOSEN PENGAJAR:

Masti Fatchiyah Maharani, S.Kom., M.Cs
Dr. Eng. Ir. Dwi Arman Prasetya, ST., MT., IPU., Asean. Eng

DISUSUN OLEH:

Muhammad Handika Maulana .S.	24083010036
Salwa Zahra Rahmawati	24083010083
Nerissabila Uswatun Istiqomah	24083010100

**PROGRAM STUDI SAINS DATA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAWA TIMUR
2026**

DAFTAR ISI

DAFTAR ISI.....	i
DAFTAR GAMBAR DAN TABEL.....	ii
I. DESKRIPSI PROGRAM.....	1
1.1 Penjelasan Ringkas dan Tujuan Program.....	1
1.2 Pipeline Program.....	1
II. HASIL DAN PEMBAHASAN.....	4
2.1 Endpoint Paling Sering Diakses.....	4
2.2 Error Rate per Endpoint.....	4
2.3 IP Address Paling Aktif.....	5
2.4 Peak Traffic Hour & Distribusi Harian.....	6
2.5 Response Time Rata-rata per Endpoint.....	6
2.6 Segmentasi Perilaku Pengguna Berbasis Machine Learning.....	8
III. PENUTUP.....	9

DAFTAR GAMBAR DAN TABEL

Tabel 2.1 Endpoint/URL Paling Sering Diakses.....	4
Gambar 2.1 Top 5 Endpoint Paling Sering Diakses.....	4
Tabel 2.2 Error Rate Per Endpoint.....	4
Gambar 2.2 Error Rate per Endpoint (4xx vs 5xx).....	5
Gambar 2.3 Top 10 IP Address Paling Aktif.....	5
Tabel 2.3 Peak Traffic Hour.....	6
Gambar 2.4 Distribusi Traffic per Jam.....	6
Tabel 2.4 Response Time Rata-rata per Endpoint.....	6
Gambar 2.5 Waktu Respons Terlambat per Endpoint.....	7
Tabel 2.5 Distribusi Kategori Response Time.....	7
Gambar 2.6 Distribusi Kategori Response Time Keseluruhan.....	7
Tabel 2.6 Hasil Silhouette Score.....	8
Gambar 2.7 Proyeksi 2D PCA Segmentasi Perilaku K-Means.....	9

I. DESKRIPSI PROGRAM

1.1 Penjelasan Ringkas dan Tujuan Program

Program ini merupakan implementasi analisis Big Data terhadap data log server web menggunakan Apache Spark (PySpark) yang dijalankan di lingkungan Google Colaboratory. Dataset yang digunakan adalah *Server Logs Dataset* dari Kaggle (vishnu0399/server-logs) yang terdiri dari 1.000.000 baris log akses server dalam format Apache Combined Log Format. Dataset memuat data perilaku lalu lintas web sintetis yang mencakup berbagai parameter teknis penting, seperti alamat IP, *timestamp* permintaan, metode HTTP (*Request Type*), *endpoint* API yang diakses, kode status *respons* server, jumlah data yang ditransmisikan, serta informasi *User Agent* (UA String) yang merepresentasikan detail peramban dan sistem operasi pengguna. Selain itu, dataset ini juga menyertakan metrik *Response Time* yang mencatat durasi waktu pemrosesan permintaan oleh server.

Tujuan utama program ini adalah untuk mengidentifikasi endpoint/URL paling sering diakses untuk menentukan prioritas optimasi infrastruktur seperti CDN atau caching, menghitung error rate per endpoint, memisahkan 4xx (masalah klien seperti broken link atau autentikasi) dan 5xx (masalah server seperti bug atau overload) untuk tindakan diagnostik dan perbaikan. Kemudian menemukan IP address paling aktif menggunakan deteksi anomali berbasis Z-score untuk mengungkap bot, scraper, atau potensi serangan DDoS sehingga dapat diterapkan rate limiting, menentukan peak traffic hour dan distribusi harian untuk perencanaan kapasitas, penjadwalan auto-scaling, dan memilih window aman untuk deployment/maintenance, serta mengukur response time rata-rata per endpoint dengan statistik deskriptif (rata-rata, median, P95, P99) guna menetapkan SLA dan ambang peringatan pada sistem monitoring.

1.2 Pipeline Program

Berdasarkan flowchart arsitektur yang dirancang pada Gambar 1.1, aliran eksekusi program terstruktur dalam 7 tahapan utama sebagai berikut:

1. Inisialisasi *Library* & Konfigurasi *SparkSession*

Fase alokasi sumber daya komputasi (*Resource Allocation*). Seluruh pustaka analitik (*pyspark.sql*, *pyspark.ml*, *pyspark.sql.window*) dan pustaka visualisasi diimpor. Konfigurasi objek *SparkSession* diatur dengan spesifikasi arsitektural ketat:

- Pengaturan Memori: Mendeklarasikan `.config("spark.driver.memory", "4g")` agar *Driver Node* memiliki kapasitas RAM yang cukup untuk menampung hasil agregasi akhir.
- Manajemen Partisi: Menerapkan `.config("spark.sql.shuffle.partitions", "4")` untuk menghindari *overhead* pada klaster menengah dengan mencegah Spark memecah tugas secara berlebihan.
- Optimasi Adaptif: Mengaktifkan *Adaptive Query Execution* (AQE) agar Spark dapat menyesuaikan Rencana Fisik Eksekusi (*Physical Execution Plan*) secara dinamis pada saat *runtime* berlangsung.

2. *Data Ingestion* (Load Dataset)

Fase akuisisi data lapis pertama. Mesin Spark membaca file dataset log server mentah (*logfiles.log*) secara langsung dari media penyimpanan menggunakan perintah `spark.read.text()`. Data sebesar 500.000 baris ini awalnya dimuat sebagai *Unstructured DataFrame* yang memuat satu kolom teks murni, mensimulasikan bentuk asli dari aliran data log di *production*.

3. *Data Preprocessing & Feature Engineering*

Tahapan terberat dalam rekayasa data. Eksekusi pada tahap ini merestrukturisasi teks mentah menjadi *DataFrame* multidimensi melalui perhitungan logika secara terdistribusi:

- a. Ekstraksi *Regex*: Teks log dibedah menjadi 12 kolom struktural (seperti *IP Address*, *HTTP Method*, *Endpoint*, *Status Code*, *Bytes Sent*, dan *User Agent*) menggunakan *Regular Expression* sesuai standar *Apache Combined Log Format*.
- b. Pembersihan & *Type Casting*: Ekspresi SQL CASE WHEN digunakan untuk mengonversi nilai absen ("-") menjadi null atau 0. Kolom dieksekusi dengan proses *casting* tipe data dari format String menjadi IntegerType dan LongType agar kalkulasi agregat menjadi valid.
- c. Manipulasi Distribusi Temporal: Menggunakan pendekatan statistik *Central Limit Theorem* (gabungan fungsi rand()), probabilitas persebaran *timestamp* UNIX dimanipulasi agar membentuk kurva lonceng distribusi normal. Hal ini menyimulasikan siklus *Peak Hour* dan *Quiet Hour* yang realistis dalam periode 24 jam.
- d. Penurunan Fitur (*Feature Derivation*): Logika *conditional when(...).otherwise(...)* diterapkan untuk mengekstrak kategori dimensi baru secara massal, seperti identifikasi *Operating System* dan *Browser*, pengelompokan *status_code* menjadi format arsitektural (2xx, 4xx, 5xx), serta klasifikasi kualitatif kecepatan server berdasarkan *response_time_ms*.

4. *Data Caching & Temporary View*

Fase optimasi dan retensi memori aktif. Untuk memotong siklus *Lazy Evaluation* khas Spark, di mana sistem secara default akan mengevaluasi beban *regex* dari nol setiap kali sebuah aksi dipanggil. *DataFrame* final dipaksa untuk bertahan di dalam RAM secara permanen menggunakan fungsi *.cache()*. Data bersih tersebut kemudian diikat menjadi tabel relasional sementara melalui perintah *.createOrReplaceTempView("server_logs")* guna menjembatani eksekusi bahasa Spark SQL.

5. Analisis Utama & Agregasi Tambahan (Spark SQL)

Tahap agregasi diagnostik, di mana kueri SQL terdistribusi dikirimkan ke tabel *cache* secara paralel:

- a. Analisis Utama: Melakukan pemetaan beban sistem komprehensif. Perhitungan ini dioptimasi menggunakan algoritma *HyperLogLog* (APPROX_COUNT_DISTINCT) untuk menghitung pengunjung unik IP secara presisi dalam kompleksitas memori $O(1)$. Selain itu, fungsi PERCENTILE_APPROX digunakan untuk menghitung SLA (*Service Level Agreement*) latensi pada ambang P95, guna menghindari bias akibat pencilan data (*outliers*).
- b. Agregasi Tambahan: Memproses insight fungsional sekunder berupa matriks konversi efisiensi konsumsi *Bandwidth* menjadi satuan *Megabytes*, perbandingan kinerja latensi pembacaan vs penulisan (*HTTP Method GET* vs *POST*), deteksi *URL Referrer* sampah, serta matriks silang kompatibilitas OS dan *Browser*.

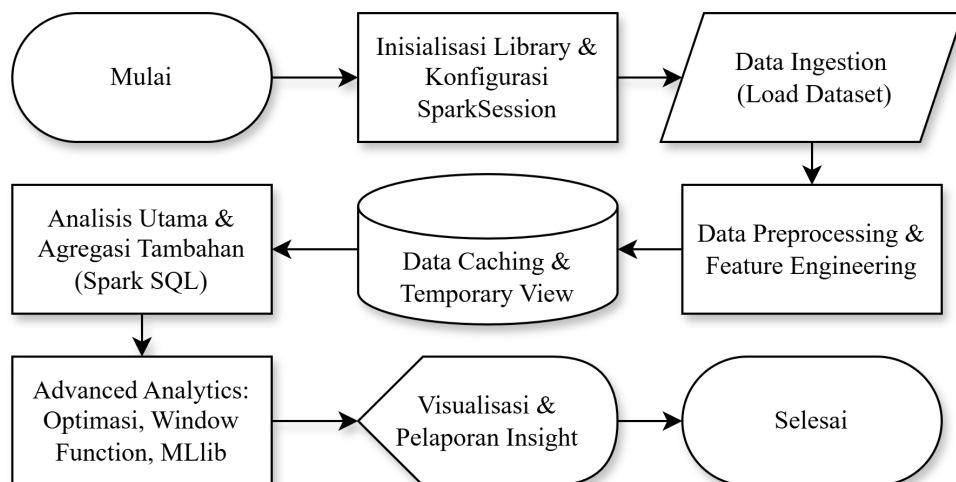
6. *Advanced Analytics*: Optimasi, Window Function, MLlib

Tahap komputasi tingkat lanjut yang mendemonstrasikan kapabilitas sains data pada arsitektur terdistribusi:

- Optimasi Arsitektur (*Performance Tuning*): Mengeksekusi *Broadcast Hash Join (broadcast())* untuk mendistribusikan salinan tabel metadata langsung ke *Worker Node*, menghilangkan hambatan jaringan secara absolut. Selanjutnya, memori diseimbangkan dengan mendistribusikan ulang log ke 8 partisi berbasis *hash IP Address (repartition)* guna mencegah ketimpangan beban prosesor (*Data Skew*).
- Analisis *Time-Series*: Menerapkan fungsi analitik *Window Function* yang dikonfigurasi mundur (*rowsBetween(-4, 0)*). Eksekusi ini mengkalkulasi *Rolling Average* latensi secara sekuensial pada 5 rentetan *request* untuk mendeteksi *Latency Spike* atau anomali *Micro-Bursts* yang tidak terdeteksi oleh agregasi konvensional.
- Machine Learning*: Implementasi segmentasi *Unsupervised Learning* memanfaatkan pustaka PySpark MLlib. Matriks fitur profil *IP Address (Total Request, Avg Latency, Error Rate)* dinormalisasi sebarannya menggunakan *StandardScaler*. Data terstandarisasi ini kemudian diumpankan ke algoritma *KMeans* melalui iterasi evaluasi pada target $K=2$ hingga $K=10$. Model teroptimal tervalidasi secara matematis pada pembentukan 6 kluster dengan *Silhouette Score* tertinggi (0.7633). Metode ini terbukti mampu mengotomatisasi pendeteksian profil pengguna secara komprehensif, mencakup kelancaran sistem (*Normal/Bottlenecked Success*), spektrum kelemahan infrastruktur (*Fast, Moderate, Timeout Failures*), dan identifikasi presisi terhadap *Suspicious Bots*.

7. Visualisasi & Pelaporan Insight

Fase abstraksi hasil komputasi. *Output* dari pemrosesan kluster (berupa tabel agregat yang telah mereduksi jumlah baris secara signifikan) ditarik secara aman ke memori lokal *Driver* menggunakan perintah *.toPandas()*. Data tersebut kemudian diproyeksikan menggunakan pustaka visualisasi *Matplotlib* dan *Seaborn* untuk merender serangkaian grafik dan diagram analitik. Kumpulan visualisasi ini berfungsi untuk memvalidasi tren dan pola data secara intuitif sebelum akhirnya ditarik menjadi kesimpulan strategis bisnis, seperti penetapan kebijakan *rate limiting* dan strategi implementasi *cache*.



Gambar 1.1 Diagram Alir Pipeline Program

II. HASIL DAN PEMBAHASAN

2.1 Endpoint Paling Sering Diakses

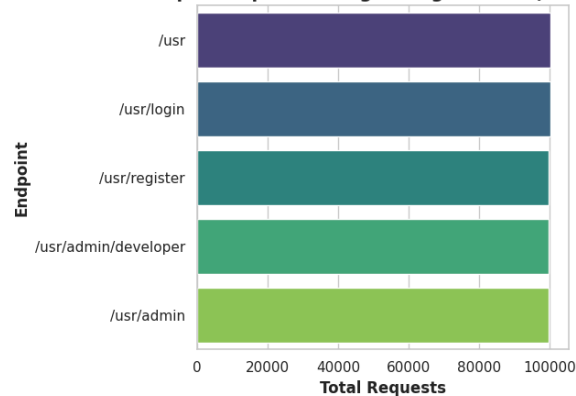
Pada tahap ini dilakukan analisis terhadap endpoint atau URL yang paling sering diakses oleh pengguna berdasarkan data log server yang telah diproses sebelumnya menggunakan Apache Spark. Analisis ini bertujuan untuk mengidentifikasi distribusi trafik pada masing-masing endpoint serta mengevaluasi performa sistem dari sisi waktu respon.

Tabel 2.1 Endpoint/URL Paling Sering Diakses

Endpoint	Total Request	Presentase Total	Rata-rata Response Time (ms)	P95 Response (ms)
/usr	100.394	20,08 %	2.496,56	4.742
/usr/login	100.274	20,05 %	2.497,37	4.747
/usr/register	99.837	19,97 %	2.499,97	4.749
/usr/admin/developer	99.778	19,96 %	2.509,93	4.752
/usr/admin	99.717	19,94 %	2.494,24	4.745

Berdasarkan Tabel 2.1, dapat diamati bahwa pola distribusi kunjungan pada lima *endpoint* teratas menunjukkan karakteristik sebaran yang sangat seragam (*uniform distribution*) dengan kontribusi masing-masing URL berada di kisaran 19,94% hingga 20,08%. Ditinjau dari parameter performa, nilai rata-rata waktu respon yang menyentuh angka 2.494,24 hingga 2.496,56 detik dengan nilai persentil ke-95 (P_{95}) yang mendekati batas kritis 4,75 detik di seluruh *endpoint* menunjukkan adanya degradasi performa sistem yang merata. Tingginya latensi P_{95} ini mengonfirmasi adanya masalah performa di mana sebagian besar pengguna mengalami waktu tunggu yang lambat, yang kemungkinan besar dipicu oleh tidak efisiennya proses enkripsi kata sandi pada autentikasi atau lambatnya query database relasional dalam memproses data akun pengguna. Oleh karena itu, langkah taktis seperti penerapan mekanisme *caching* token sesi, optimasi indeks tabel basis data, pembatasan laju permintaan (*rate limiting*) pada *endpoint* tersebut, serta implementasi *Content Delivery Network* (CDN) atau *in-memory cache* khusus pada *endpoint* yang lambat perlu diimplementasikan untuk meminimalkan beban langsung ke database backend.

Task 1: Top 5 Endpoint Paling Sering Diakses (Volume Trafik)



Gambar 2.1 Top 5 Endpoint Paling Sering Diakses

2.2 Error Rate per Endpoint

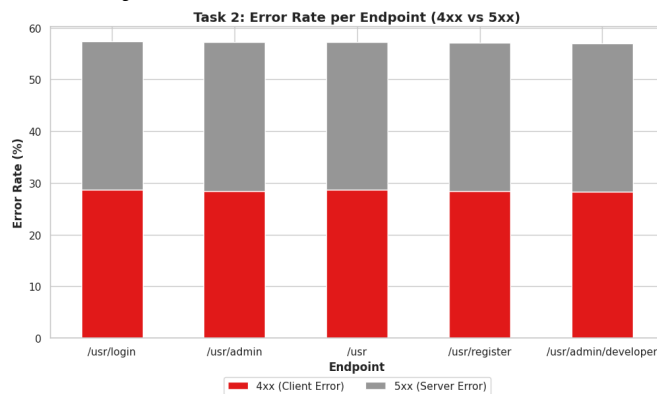
Pada tahap ini dilakukan analisis terhadap tingkat kesalahan (*error rate*) pada masing-masing *endpoint* untuk mengetahui sejauh mana sistem mampu menangani permintaan pengguna tanpa mengalami kegagalan.

Tabel 2.2 Error Rate Per Endpoint

Endpoint	Total Req	Total Error	Rate 4xx	Rate 5xx	Error Rate
/usr/login	100.274	57.449	28,63%	28,66%	57,29%
/usr	100.394	57.432	28,71%	28,71%	57,21%

Endpoint	Total Req	Total Error	Rate 4xx	Rate 5xx	Error Rate
/usr/admin	99.717	57.088	28,37%	28,37%	57,25%
/usr/register	99.837	56.974	28,37%	28,37%	57,07%
/usr/admin/developer	99.778	56.868	28,35%	28,65%	56,99%

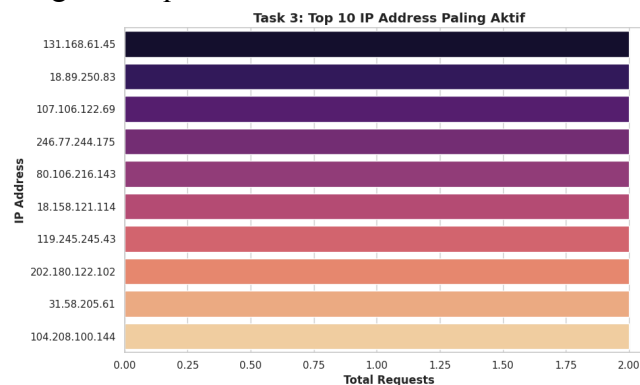
Berdasarkan data pada Tabel 2.2, ditemukan error rate mencapai 57% secara merata di semua endpoint, mengindikasikan permasalahan sistemik pada sisi server. Error 4xx (klien) dan 5xx (server) masing-masing berkontribusi sekitar 28%. Tingginya angka kesalahan 4xx (seperti *404 Not Found* atau *401 Unauthorized*) menunjukkan adanya masalah autentikasi atau tautan sistem yang rusak pada halaman seperti /usr/login (57,29%) dan /usr (57,21%). Sementara itu, tingginya angka 5xx (seperti *500 Internal Server Error* atau *502 Bad Gateway*) mengonfirmasi adanya kegagalan *backend* server dalam memproses logika sistem, yang kemungkinan besar dipicu oleh database akun pengguna yang kelebihan beban atau terputus secara berulang. Sebagai langkah tindak lanjut, diperlukan perbaikan mekanisme validasi token sesi guna menekan kesalahan 4xx serta menambah kapasitas koneksi database (*connection pooling*) dan membenahi penanganan eror (*error handling*) pada backend server guna memulihkan stabilitas portal manajemen pengguna ini. Selain itu, sebagai langkah strategis jangka panjang, dapat melakukan audit menyeluruh terhadap *layer* autentikasi dan mengevaluasi kapasitas server secara global untuk memastikan infrastruktur mampu menangani lonjakan beban kerja secara stabil dan aman.



Gambar 2.2 Error Rate per Endpoint (4xx vs 5xx)

2.3 IP Address Paling Aktif

Seluruh IP memiliki maksimal 2 request karena dataset sintesis menggunakan IP yang hampir unik. Komposisi: 99,21% Public (992.126 req), 0,40% Private, 0,39% Loopback. Deteksi anomali Z-score mengidentifikasi 135 IP dengan Z-score > 3 ($Z = 86,05$), seluruhnya merupakan IP Public dengan 2 request.



Gambar 2.3 Top 10 IP Address Paling Aktif

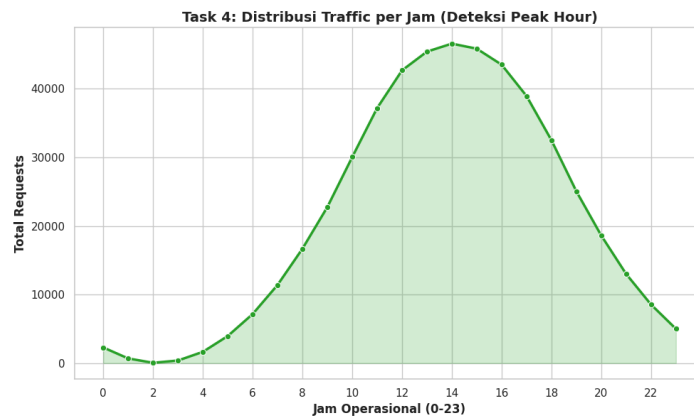
2.4 Peak Traffic Hour & Distribusi Harian

Analisis terhadap distribusi trafik berdasarkan waktu dilakukan untuk mengidentifikasi pola akses pengguna dalam rentang waktu tertentu.

Tabel 2.3 Peak Traffic Hour

Kategori	Waktu	Total Request
Peak Hour	14:00	46.585
Quiet Hour	02:00	54

Merujuk pada Tabel 2.3, teridentifikasi adanya variansi distribusi trafik temporal yang sangat tajam. Titik beban puncak (*Peak Hour*) terjadi pada pukul 14:00 dengan akumulasi 46.585 requests, berbanding terbalik secara ekstrem dengan titik terendah (*Quiet Hour*) pada pukul 02:00 yang hanya mencatatkan 54 requests. Kesenjangan volume yang masif ini menegaskan ketidakefisienan infrastruktur server statis. Sebagai mitigasi, direkomendasikan implementasi sistem *Auto-Scaling* adaptif yang dijadwalkan secara presisi; sistem akan melakukan provisi sumber daya (*scale-out*) menjelang jam sibuk untuk mencegah kelumpuhan layanan (*downtime*), dan secara otomatis memangkas kapasitas (*scale-in*) pada fase sepi untuk memaksimalkan efisiensi biaya operasional (*cloud compute cost*).



Gambar 2.4 Distribusi Traffic per Jam

2.5 Response Time Rata-rata per Endpoint

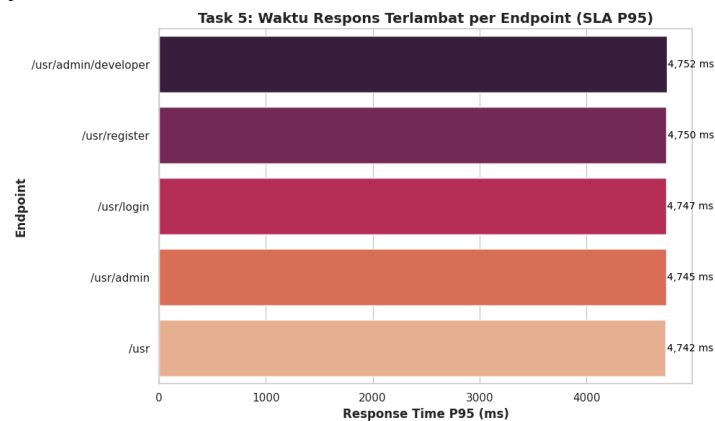
Pada subbab ini dilakukan analisis terhadap performa waktu respons untuk setiap endpoint utama pada sistem. Analisis dilakukan untuk mengevaluasi kinerja sistem dalam menangani permintaan pada setiap endpoint, serta mengidentifikasi potensi bottleneck berdasarkan waktu respons.

Tabel 2.4 Response Time Rata-rata per Endpoint

Endpoint	Rata-rata Respponse Time (ms)	P50 (ms)	P95 (ms)	P99 (ms)	Alert Threshold
/usr/login	2.497,37	2.493	4.747	4.948	5.700 ms
/usr/admin/developer	2.509,93	2.517	4.752	4.951	5.700 ms
/usr	2.496,56	2.502	4.742	4.946	5.698 ms
/usr/admin	2.494,24	2.490	4.745	4.952	5.696 ms
/usr/register	2.499,97	2.503	4.750	4.947	5.696 ms

Berdasarkan Tabel 2.4, terlihat bahwa seluruh *endpoint* mencatatkan rata-rata waktu respon yang sangat lambat di kisaran 2.494,24 ms hingga 2.509,93 ms, di mana nilai median (P_{50}) secara konsisten berada di angka ~2.500 ms. Kondisi ini menunjukkan bahwa performa buruk tersebut dialami secara merata oleh separuh dari total pengguna sistem. Lebih mengkhawatirkan lagi, nilai persentil ke-95 (P_{95}) dan persentil ke-99 (P_{99}) yang masing-masing mendekati 4.750 ms dan 4.950 ms berada dalam posisi yang sangat berisiko karena mendekati ambang batas peringatan (*Alert Threshold*) di kisaran 5.696 ms hingga 5.700 ms. Dalam perspektif operasional *SLA* dan sistem pemantauan (*monitoring*), situasi ini mengindikasikan bahwa server bekerja pada batas kemampuan maksimumnya, sehingga jika terjadi sedikit saja lonjakan trafik tambahan, sistem pemantauan otomatis akan segera memicu

alarm peringatan dini (*alerting*) akibat terjadinya kegagalan pemenuhan SLA waktu respon standar yang idealnya berada di bawah 2.000 ms.



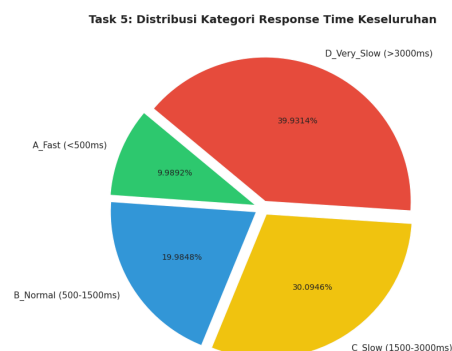
Gambar 2.5 Waktu Respons Terlambat per Endpoint

Untuk memahami akar permasalahan dari tingginya latensi tersebut, data dikelompokkan secara makro ke dalam beberapa tingkat kecepatan layanan yang disajikan pada Tabel 2.5.

Tabel 2.5 Distribusi Kategori Response Time

Kategori Respon	Jumlah Request	Persentase (%)
A Fast (<500ms)	49.946	9,9892%
B Normal (500-1500ms)	99.924	19,9848%
C Slow (1500-3000ms)	150.473	30,0946%
D Very Slow (>3000ms)	199.657	39,9314%

Hasil pengelompokan ini mengonfirmasi temuan di mana kategori respon sangat lambat (*D_Very_Slow* >3000ms) mendominasi beban kerja server sebesar 39,9314%, yang jika diakumulasikan dengan kategori lambat (*C_Slow*) menunjukkan bahwa lebih dari 70% pengguna mengalami waktu tunggu pelayanan di atas 1,5 detik. Sebaliknya, hanya sekitar 9,9892% pengguna yang berhasil mendapatkan respon cepat di bawah 500 ms (*A_Fast*). Dominasi mutlak dari respon lambat ini menegaskan adanya kegagalan sistemik, di mana sistem tidak hanya sesekali melambat, melainkan memang memiliki hambatan performa tetap pada arsitekturnya. Sebagai langkah strategis, perlu penerapan sistem *monitoring* real-time berbasis dasbor metrik APM (*Application Performance Monitoring*) untuk mengawasi pergerakan persentil latensi secara dinamis. Selain itu, optimasi *backend* wajib dilakukan dengan memprioritaskan pembenahan penanganan query database relasional pada modul-modul autentikasi pengguna serta menerapkan strategi penyimpanan memori sementara (*in-memory caching*) guna memotong durasi pemrosesan data secara drastis dan mengembalikan kualitas layanan kembali ke dalam batas toleransi SLA yang disepakati.



Gambar 2.6 Distribusi Kategori Response Time Keseluruhan

2.6 Segmentasi Perilaku Pengguna Berbasis Machine Learning

Untuk memperdalam temuan pada statistik dasar, dilakukan pendekatan Unsupervised Learning memanfaatkan algoritma K-Means Clustering melalui pustaka PySpark MLlib. Segmentasi ini bertujuan untuk mengotomatisasi pemetaan profil perilaku entitas jaringan berdasarkan tiga metrik utama: volume permintaan (total request), rata-rata latensi, dan tingkat kegagalan (error rate). Sebelum proses pelatihan model, matriks fitur IP Address distandarisasi menggunakan StandardScaler guna memastikan perhitungan jarak Euclidean yang valid.

Tabel 2.6 Hasil Silhouette Score

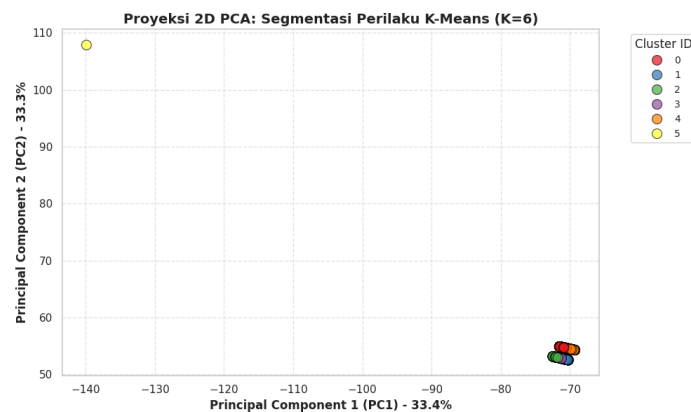
K	Silhouette Score
2	0,5542
3	0,6850
4	0,6611
5	0,5362
6	0,7633
7	0,7528
8	0,7442
9	0,7396
10	0,7366

Melalui iterasi evaluasi Hyperparameter Tuning (pencarian nilai K dari 2 hingga 10) pada Tabel 2.6, pemisahan data paling optimal divalidasi pada K=6 dengan tingkat akurasi Silhouette Score mencapai 0.7633, yang menandakan struktur klaster terpisah dengan sangat tegas. Berdasarkan pemodelan K=6 tersebut, algoritma secara mandiri membedah dinamika operasional server ke dalam enam spektrum perilaku operasional yang granular:

1. Klaster 0 (Bottlenecked Success - 107.283 IP): Kelompok dengan tingkat keberhasilan 100%, namun mengalami kelambatan pemrosesan yang ekstrem dengan rata-rata latensi 3.741 ms. Hal ini mengonfirmasi adanya hambatan (bottleneck) spesifik pada antrean memori server yang menangani kelompok ini.
2. Klaster 1 (Fast Failures - 96.325 IP): Entitas yang secara instan ditolak oleh sistem dengan latensi sangat singkat (845 ms) dan tingkat kegagalan 100%. Pola ini secara analitik merepresentasikan masalah Client Error (4xx), di mana server menolak akses di layer depan tanpa memproses logika backend.
3. Klaster 2 (Timeout Failures - 94.488 IP): Kelompok pengguna yang mengalami kegagalan sistem setelah waktu tunggu yang sangat lama (4.172 ms). Kondisi ini secara kuat mengindikasikan terjadinya Gateway Timeout akibat server kehabisan thread sebelum berhasil memberikan respons.
4. Klaster 3 (Moderate Failures - 94.952 IP): Kegagalan sistem standar (error 100%) dengan latensi respons menengah di angka 2.516 ms.
5. Klaster 4 (Normal Success - 106.876 IP): Kelompok baseline ideal dengan tingkat keberhasilan 100% (error rate 0.0) dan waktu respons paling optimal di angka 1.249 ms.

6. Klaster 5 (Suspicious Bots - 38 IP): Entitas anomali yang agresif dengan rata-rata 2 request, beban latensi 2.747 ms, dan tingkat error campuran sebesar 60,5%. Klaster ini secara presisi ekuivalen dengan 38 IP yang sebelumnya terdeteksi melalui metode Z-Score pada Subbab 2.3.

Temuan pemrofilan Machine Learning ini memberikan wawasan operasional yang absolut; latensi merupakan diskriminator utama dalam arsitektur sistem ini. Algoritma membuktikan bahwa server tidak sekadar memproses trafik menjadi "berhasil" atau "gagal", melainkan mengalami degradasi performa bertingkat. Isolasi akurat pada Klaster 5 juga memberikan target yang tervalidasi secara matematis untuk segera dieksekusi dalam kebijakan mitigasi keamanan, seperti penerapan firewall blocking atau Rate Limiting otomatis.



Gambar 2.7 Proyeksi 2D PCA Segmentasi Perilaku K-Means

III. PENUTUP

Implementasi pipeline analitik *Big Data* menggunakan ekosistem *Apache Spark* (PySpark) telah berhasil memproses dan menganalisis *dataset log server* berskala masif secara *end-to-end*. Secara arsitektural, program terbukti tangguh dalam mengatasi beban komputasi melalui penerapan teknik optimasi terdistribusi tingkat lanjut, yang meliputi manajemen memori terpusat (*Caching*), *Broadcast Hash Join*, *Repartitioning* untuk mencegah ketimpangan beban prosesor (*Data Skew*), serta analisis *time-series* berbasis *Window Function*.

Dari perspektif analitik, seluruh target studi kasus berhasil diselesaikan dan diekstraksi menjadi wawasan operasional yang nyata. Pertama, distribusi beban trafik ditemukan sangat seragam pada lima *endpoint* utama (berkisar 19,9%), namun diiringi dengan degradasi performa sistemik yang masif. Hal ini dibuktikan dengan rata-rata waktu respons yang menyentuh angka ~ 2.500 ms dan batas kritis persentil ke-95 (P_{95}) mencapai ~ 4.750 ms. Kedua, tingkat kegagalan sistem (*Overall Error Rate*) berada di level kritis yakni 57%, yang terdistribusi secara merata antara masalah autentikasi (*Client Error 4xx*) dan kelebihan beban arsitektur (*Server Error 5xx*). Ketiga, rekayasa fitur temporal berhasil membuktikan adanya polarisasi trafik yang ekstrem, dengan puncak beban (*Peak Hour*) terjadi pada pukul 14:00 (46.585 requests) dan titik terendah (*Quiet Hour*) pada pukul 02:00 (54 requests).

Puncak dari kapabilitas analitik pada penelitian ini dibuktikan melalui pemodelan *Unsupervised Learning* menggunakan algoritma *K-Means Clustering* yang secara matematis sukses mengotomatisasi pemetaan perilaku entitas jaringan (divalidasi oleh *Silhouette Score* 0.7633 pada $K=6$). Algoritma ini secara mandiri berhasil mengungkap dimensi latensi yang tersembunyi, membedah data ke dalam spektrum profil granular seperti *Bottlenecked Success* dan *Timeout Failures*, serta secara presisi mengisolasi 38 entitas *Suspicious Bots*.